

C7A

CONTRÔLE CONTINU DE PROGRAMMATION SYSTÈME 3GI

1. Define and give the difference between a process and a thread ? 2 pts
2. Quelles commandes sur les systèmes d'exploitation Linux sont utilisées pour :
 - afficher la liste de tous les processus s'exécutant dans le système ? 0.75pt
 - afficher uniquement la liste des processus de l'utilisateur courant ? 0.75pt
3. Name the different process states. 2pts
4. Define signals and name three predefined signal in UNIX. 2pts
5. Décrivez ce qui se passe à l'intérieur du système d'exploitation quand la séquence CTRL+ALT+DELETE est entrée au clavier. 1.5pts (sur Windows)
6. En exécutant la commande permettant d'afficher la liste de tous les fichiers et répertoires présents dans son répertoire courant (le Système d'exploitation utilisé est Linux), l'utilisateur John repère un fichier intitulé Test3GI1 avec les permissions 4654. John appartient au groupe G1 avec d'autres utilisateurs tels que Malica et Alfred.
 - a- Quelles actions John peut-il entreprendre sur le fichier Test3GI1 ? 1 pt
 - b- Quelles actions Malica peut-elle entreprendre sur le fichier Test3GI1 ? 1pt
 - c- Quelles actions Eric qui n'est pas membre du groupe G1 peut-il entreprendre sur le fichier Test3GI1 ? 1pt
 - d- Après la commande `chmod go-rx`, quelles actions Alfred et John seront-ils capable d'entreprendre sur ce fichier? expliquer l'action de cette commande. 1pt
7. Write a code in which a parent process creates a child process which will also create its child process. Each child process should display its pid and the pid of its father. Each Father should display the pid of its child. 3 pts
N.B: This program should be written in C-Language and will be run on a Linux operating System. You should insert as many comments as necessary to explain your code and the outputs.
8. Définir et expliquer la différence entre un *Named pipe* et un *Unnamed pipe*. 2pts
9. Expliquer le programme suivant 2pts

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<string.h>
#include<sys/wait.h>
```

```
int main()
{   int fd1[2];
```

```

int fd2[2];
char fixed_str[] = "forgeeks.org";
char input_str[100];
pid_t p;
if (pipe(fd1)==-1)
{
    fprintf(stderr, "Pipe Failed" );
    return 1;
}
if (pipe(fd2)==-1)
{
    fprintf(stderr, "Pipe Failed" );
    return 1;
}
scanf("%s", input_str);
p = fork();
if (p < 0)
{
    fprintf(stderr, "fork Failed" );
    return 1;
}

else if (p > 0)
{
    char concat_str[100];
    close(fd1[0]);
    write(fd1[1], input_str, strlen(input_str)+1);
    close(fd1[1]);
    wait(NULL);
    close(fd2[1]);
    read(fd2[0], concat_str, 100);
    printf("Concatenated string %s\n", concat_str);
    close(fd2[0]);
}
else
{
    close(fd1[1]);
    char concat_str[100];
    read(fd1[0], concat_str, 100);
    int k = strlen(concat_str);
    int i;
    for (i=0; i<strlen(fixed_str); i++)
        concat_str[k++] = fixed_str[i];
    concat_str[k] = '\0';
    close(fd1[0]);
    close(fd2[0]);
    write(fd2[1], concat_str, strlen(concat_str)+1);
    close(fd2[1]);

    exit(0);
}
}

```